BACSE202- Database Systems

by Pratik Ambastha 25BCE0337

PL/SQL Assignment 7

Part A: Self-Learning Task (5 Marks)

1. Introduction to PL/SQL and Differences Between SQL and PL/SQL

PL/SQL (Procedural Language/SQL) is Oracle's procedural extension to SQL. It combines the data manipulation power of SQL with the procedural constructs of a programming language (variables, loops, conditions, exception handling). PL/SQL is executed as a single block on the database server, reducing network traffic and improving performance.

Feature	SQL	PL/SQL
Type	Declarative	Procedural
Execution	Executes one statement at a time	Executes an entire block at once
Variables	Not supported	Supported
Control structures	Not supported	IF, LOOP, WHILE, FOR
Exception handling	Not supported	Supported
Reusability	Limited	High (procedures, functions, packages)
Interaction with DB	Direct	Embedded SQL within procedural code

Example:

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
END;
/
```

2. Structure of a PL/SQL Block

A PL/SQL block has four sections:

```
DECLARE     -- (Optional) Variables, constants, cursors
BEGIN       -- (Mandatory) Executable statements
EXCEPTION   -- (Optional) Error handling
END;        -- (Mandatory) End of block
/
```

Example:

```
DECLARE
   v_msg VARCHAR2(50) := 'Hello from PL/SQL';
BEGIN
   DBMS_OUTPUT.PUT_LINE(v_msg);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An error occurred');
END;
/
```

3. Declaration and Use of Variables, Constants, and Data Types

- **Variables** store values that can change during execution.
- **Constants** store values that cannot change (declared with `CONSTANT`).
- Common data types: `NUMBER`, `VARCHAR2`, `DATE`, `BOOLEAN`, `%TYPE`.

Syntax:

```
DECLARE
   v_name   VARCHAR2(30) := 'John';
   v_age    NUMBER := 25;
   c_pi     CONSTANT NUMBER := 3.14159;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);
   DBMS_OUTPUT.PUT_LINE('PI: ' || c_pi);
END;
/
```

4. Conditional Statements: IF, IF-ELSE, ELSIF

Syntax:

```
IF condition THEN
   statements;
ELSIF condition THEN
   statements;
ELSE
```

```
   statements;
END IF;
```

Example:

```
DECLARE
   v_sales NUMBER := 250;
BEGIN
   IF v_sales > 300 THEN
      DBMS_OUTPUT.PUT_LINE('Excellent');
   ELSIF v_sales > 200 THEN
      DBMS_OUTPUT.PUT_LINE('Good');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Needs Improvement');
   END IF;
END;
/
```

---

5. Iterative Statements: FOR LOOP and WHILE LOOP

FOR LOOP — runs a fixed number of times:

```
BEGIN
   FOR i IN 1..5 LOOP
      DBMS_OUTPUT.PUT_LINE('Number: ' || i);
   END LOOP;
END;
/
```

WHILE LOOP — runs while a condition is true:

```
DECLARE
   i NUMBER := 1;
BEGIN
   WHILE i <= 5 LOOP
      DBMS_OUTPUT.PUT_LINE('Number: ' || i);
      i := i + 1;
   END LOOP;
END;
/
```

6. Using SELECT INTO to Retrieve Data

`SELECT INTO` fetches a single row from a table into PL/SQL variables. It must return exactly one row, or an exception is raised.

Syntax:

```
SELECT column1, column2
INTO   variable1, variable2
FROM   table_name
WHERE  condition;
```

Example:

```
DECLARE
   v_name  SALESREPS.NAME%TYPE;
   v_sales SALESREPS.SALES%TYPE;
BEGIN
   SELECT NAME, SALES
   INTO   v_name, v_sales
   FROM   SALESREPS
   WHERE  EMPL_NUM = 101;

   DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Sales: ' || v_sales);
END;
/
```

7. Displaying Output Using DBMS_OUTPUT.PUT_LINE

`DBMS_OUTPUT.PUT_LINE` prints a line of text to the console. Requires `SET SERVEROUTPUT ON;`.

Example:

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('This is a test output.');
END;
/
```

8. Basic Exception Handling in PL/SQL

Exceptions handle runtime errors gracefully. Common predefined exceptions: `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `ZERO_DIVIDE`, `OTHERS`.

Syntax:

```
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      handler_statements;
   WHEN OTHERS THEN
      handler_statements;
END;
```

Example:

```
DECLARE
   v_name SALESREPS.NAME%TYPE;
BEGIN
   SELECT NAME INTO v_name
   FROM   SALESREPS
   WHERE  EMPL_NUM = 999;

   DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Salesperson not found.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
END;
/
```

Part B: Programming Questions (5 Marks)

Q1. Sum of Two Numeric Variables (1 Mark)

Problem: Declare two numeric variables, assign them values of 100 and 200, calculate their sum, and display the result.

```
SET SERVEROUTPUT ON;

DECLARE
   v_num1 NUMBER := 100;
   v_num2 NUMBER := 200;
   v_sum  NUMBER;
BEGIN
   v_sum := v_num1 + v_num2;
   DBMS_OUTPUT.PUT_LINE('Sum of ' || v_num1 || ' and ' || v_num2 || ' is: ' ||
v_sum);
END;
/
```

Expected Output:

```
Sum of 100 and 200 is: 300
```

Q2. Retrieve Name and Sales of Salesperson 101 (1 Mark)

Problem: Retrieve the name and sales of salesperson number 101 from the `SALESREPS` table and display the retrieved values.

```
SET SERVEROUTPUT ON;

DECLARE
   v_name  SALESREPS.NAME%TYPE;
   v_sales SALESREPS.SALES%TYPE;
BEGIN
   SELECT NAME, SALES
   INTO   v_name, v_sales
   FROM   SALESREPS
   WHERE  EMPL_NUM = 101;

   DBMS_OUTPUT.PUT_LINE('Salesperson Name : ' || v_name);
```

```
   DBMS_OUTPUT.PUT_LINE('Sales           : ' || v_sales);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Salesperson 101 not found.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Expected Output:

```
Salesperson Name : Dan Roberts
Sales            : 305673
```

Q3. Check Quota Achievement for Salesperson 102 (1 Mark)

Problem: Retrieve the sales and quota of salesperson number 102. Display whether the salesperson has achieved the assigned quota.

```
SET SERVEROUTPUT ON;

DECLARE
   v_name  SALESREPS.NAME%TYPE;
   v_sales SALESREPS.SALES%TYPE;
   v_quota SALESREPS.QUOTA%TYPE;
BEGIN
   SELECT NAME, SALES, QUOTA
   INTO   v_name, v_sales, v_quota
   FROM   SALESREPS
   WHERE  EMPL_NUM = 102;

   DBMS_OUTPUT.PUT_LINE('Salesperson : ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Sales       : ' || v_sales);
   DBMS_OUTPUT.PUT_LINE('Quota       : ' || v_quota);

   IF v_sales >= v_quota THEN
      DBMS_OUTPUT.PUT_LINE('Status: Quota ACHIEVED.');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Status: Quota NOT achieved.');
   END IF;
```

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Salesperson 102 not found.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Expected Output:

```
Salesperson : Sue Smith
Sales       : 474050
Quota       : 350000
Status: Quota ACHIEVED.
```

Q4. Display Numbers 1 to 10 Using FOR LOOP (1 Mark)

Problem: Display the numbers from 1 to 10 using a FOR LOOP.

```
SET SERVEROUTPUT ON;

BEGIN
   DBMS_OUTPUT.PUT_LINE('Numbers from 1 to 10:');
   FOR i IN 1..10 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
   END LOOP;
END;
/
```

Expected Output:

```
Numbers from 1 to 10:
1
2
3
4
5
6
7
8
```

```
9
10
```

Q5. Customer Lookup with Exception Handling (1 Mark)

Problem: Accept a customer number and display the corresponding company name and credit limit. If the customer number does not exist in the `CUSTOMERS` table, display an appropriate message using exception handling.

```
SET SERVEROUTPUT ON;

DECLARE
   v_cust_num     CUSTOMERS.CUST_NUM%TYPE := &customer_number;
   v_company      CUSTOMERS.COMPANY%TYPE;
   v_credit_limit CUSTOMERS.CREDIT_LIMIT%TYPE;
BEGIN
   SELECT COMPANY, CREDIT_LIMIT
   INTO   v_company, v_credit_limit
   FROM   CUSTOMERS
   WHERE  CUST_NUM = v_cust_num;

   DBMS_OUTPUT.PUT_LINE('Customer Number : ' || v_cust_num);
   DBMS_OUTPUT.PUT_LINE('Company Name    : ' || v_company);
   DBMS_OUTPUT.PUT_LINE('Credit Limit    : ' || v_credit_limit);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Customer number ' || v_cust_num || ' does not exist in
the CUSTOMERS table.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Sample Run 1 (existing customer):

```
Enter value for customer_number: 2111
Customer Number : 2111
Company Name    : JCP Inc.
Credit Limit    : 50000
```

Sample Run 2 (non-existing customer):

```
Enter value for customer_number: 9999
Customer number 9999 does not exist in the CUSTOMERS table.
```